

P2769R2

get_element customization point object

Published Proposal, 2024-06-25

This version:

<https://wg21.link/P2769R2>

Authors:

[Ruslan Arutyunyan](#) (Intel)

[Alexey Kukanov](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper introduces a CPO to read elements of tuple like objects, and uses it to define a generic *tuple-like* concept.

Table of Contents

1 Motivation

- 1.1 Allowing user-defined tuples, or "true" motivation :)
- 1.2 The original motivating use case
 - 1.2.1 Projections-based alternative
 - 1.2.2 The desired approach
 - 1.2.3 Why not `std::ranges::views::elements`
- 1.3 Usefulness with `zip_view`
- 1.4 Radix sort use case

2 Proposed API

- 2.1 Possible implementation
- 2.2 *tuple-like* concept
 - 2.2.1 *tuple-like* concept generalization with `get_element`

3 Design considerations

- 3.1 What could be done to use `std::ranges::get` name

4 Connections with other papers

- 4.1 Connection with [P2547R1]
- 4.2 Connection with [P2141R1]

5 Further work

6 Formal wording

- 6.1 Modify Concept *tuple-like* [`tuple.like`]
- 6.2 Modify Header `<tuple>` synopsis [`tuple.syn`]
- 6.3 Add the following sections into [`tuple`]
- 6.4 Add the following wording into [`tuple.cust.get_elem`]
- 6.5 Add feature test macro to the end of [`version.syn`]

- 6.6 Modify `tuple` construct [`tuple.cnstr`]
- 6.7 Modify `tuple` assignment [`tuple.assign`]
- 6.8 Modify `tuple_cat` in tuple creation [`tuple.creation`]
- 6.9 Modify `apply` in [`tuple.apply`]
- 6.10 Modify relation operators in [`tuple.rel`]
- 6.11 Modify [`range.elements.iterator`]

7 Revision history

- 7.1 R1 => R2
- 7.2 R0 => R1

8 Polls

- 8.1 SG9 polls, Issaquah 2023
- 8.2 Library Evolution Telecon 2024-01-23

9 Acknowledgements

References

Informative References

§ 1. Motivation

§ 1.1. Allowing user-defined tuples, or "true" motivation :)

The section addresses the LEWG feedback. While we always had in mind that with `get_element` facility we can make *tuple-like* generic enough, we assumed it to be developed as a separate proposal. However, based on the LEWG poll (§ 8.2 Library Evolution Telecon 2024-01-23), the `get_element` paper should enable user-defined tuples to make its motivation strong enough.

So, let's go to the problem statement. Today the C++ standard defines *tuple-like* types as only 5 types from `std` namespace:

- `std::tuple`
- `std::pair`
- `std::array`
- `std::complex` (allowed since C++26)
- `std::ranges::subrange`

That sounds like a huge limitation for a generic library because, in principle, user-defined types could be treated like tuples. `std::tuple_size` and `std::tuple_element` are already customizable by users. The problematic part is `std::get` which is not a customization point.

Furthermore, there is already partial support for user-defined tuples in the language. For example the *structured binding* language feature has special rules for finding a `get` function for an arbitrary type (in a nutshell, either as a non-static-member-function or by argument-dependent lookup).

Unfortunately, rules are different in different places in the standard today. For example, *has-tuple-element* exposition-only concept for `elements_view` allows only the 5 *tuple-like* types listed above and does not consider user-defined types.

[P2165R4] added constraints for existing API (like `std::apply`, `std::tuple_cat`, etc.) to take *tuple-like* and also provides better compatibility between *tuple-like* objects by adding extra APIs, which is great. The unfortunate part of the story, however, is that the mentioned APIs are still limited by the definition of the *tuple-like* concept (5 standard types).

Since the proposed `get_element` API is a customization point object we can use it to extend the tuple protocol to user-defined types, and since this facility is using ADL lookup for `get` it is not going to be a breaking change.

For the following (simplified) code snippet:

```
namespace user {

template <typename T, typename U>
struct my_tuple_like
{
public:
    my_tuple_like(T tt, U uu) : t(tt), u(uu) {}
private:
    T t;
    U u;

    template <std::size_t I>
    friend auto get(my_tuple_like<T, U> t_like)
    {
        static_assert (I == 0 || I == 1);
        if constexpr (I == 0)
            return t_like.t;
        else if (I == 1)
            return t_like.u;
    }
};

} // namespace user

namespace std {

template <typename T, typename U>
struct tuple_size<user::my_tuple_like<T, U>> : std::integral_constant<std::size_t, 2> {};

template <typename T, typename U>
struct tuple_element<0, user::my_tuple_like<T, U>> {
    using type = T;
};

template <typename T, typename U>
struct tuple_element<1, user::my_tuple_like<T, U>> {
    using type = U;
};

} // namespace std
```

please see the Before-After table

Before	After
<pre>auto [x, y] = user::my_tuple_like{3,3}; // This code does not compile // std::apply([](auto x, auto y) { // return x + y; // }, user::my_tuple_like{3,3});</pre>	<pre>auto [x, y] = user::my_tuple_like{3,3}; // Works fine, assuming that std::apply uses std::get_element std::apply([](auto x, auto y) { return x + y; }, user::my_tuple_like{3,3});</pre>

Of course, `std::apply` is just an example. `my_tuple_like` would work with any API that supports *tuple-like* types.

§ 1.2. The original motivating use case

Having `std::pair`, `std::tuple` and other *tuple-like* objects as the value types for the algorithms creates a plenty of opportunities. With special views, such as `std::ranges::elements_view`, we can specify which tuple elements to access

when iterating over collections of such objects. However, we cannot easily use a predicate to make a decision based on only some of tuple elements, for example keys or values.

Let's consider the following example:

```
std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}};
std::ranges::sort(v, [] (auto x, auto y)
{
    // key-based sorting
    return std::get<0>(x) < std::get<0>(y);
});
```

As we can see, users should spell some extra syntax out to achieve the necessary goal, comparing to what is described in [§ 1.2.2 The desired approach](#). The example above can be considered simplified; in real practice users might also need to think of e.g. adding references to lambda parameters to avoid copying.

The code above can be rewritten with *structured binding*:

```
std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}};
std::ranges::sort(v, [] (auto x, auto y)
{
    // key-based sorting
    auto [key1, value1] = x;
    auto [key2, value2] = y;
    return key1 < key2;
});
```

Though one could say that it makes code simpler or at least more readable, on the other hand, its syntax forces the programmer to give names to otherwise unneeded variables, which is often considered a bad practice.

With [\[P2169R3\]](#) the situation with unused variables for structured binding becomes better but still might require the user to write a quite amount of underscores depending on the use case:

```
std::vector<std::tuple<int, int, int, int>> v{{3,1,1,1},{2,4,4,4},{1,7,7,7}};
std::ranges::sort(v, [] (auto x, auto y)
{
    // key-based sorting
    auto [key1, _, _, _] = x;
    auto [key2, _, _, _] = y;
    return key1 < key2;
});
```

§ 1.2.1. Projections-based alternative

Projections provide another option to achieve the same behavior:

```
std::ranges::sort(v, std::less{}, [] (auto x)
{
    // key-based sorting
    return std::get<0>(x);
});
```

A variant that properly handles references would use a generic lambda:

```
[] (auto&& x) -> auto&&
{
    // key-based sorting
    return std::get<0>(std::forward<decltype(x)>(x));
}
```

While this code achieves the desired result, it requires more syntactic boilerplate (lambda, forwarding etc.) than the useful code.

§ 1.2.2. The desired approach

The nicest way to get what we want would be:

```
// The code that does not work because std::get is not fully instantiated
std::ranges::sort(v, std::less{}, std::get<0>);
```

But it doesn't work because `std::get` is a function template, and one cannot pass function templates as arguments without instantiating them.

§ 1.2.3. Why not `std::ranges::views::elements`

The necessary result cannot be achieved with `std::ranges::views::elements`, which would apply the filter for all operations on the input data, including element swap (for sort algorithm), while we need it to be only be applied for the comparator.

std::ranges::views::elements	Des
<pre>std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}}; // views::keys is an alias to views::elements std::ranges::sort(v std::ranges::views::keys, [](auto x, auto y) { return x < y; }); for (auto& x : v) { auto [key, val] = x; std::cout << "Key = " << key << ", Value = " << val << std::endl; } // Output (only keys are sorted): // Key = 1, Value = 1 // Key = 2, Value = 4 // Key = 3, Value = 7</pre>	<pre>std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}}; std::ranges::sort(v, [](auto x, auto y) { return std::get<0>(x) < std::get<0>(y); }); for (auto& x : v) { auto [key, val] = x; std::cout << "Key = " << key << ", Value = " << val << std::endl; } // Output (values are sorted by key): // Key = 1, Value = 7 // Key = 2, Value = 4 // Key = 3, Value = 1</pre>

§ 1.3. Usefulness with `zip_view`

With `std::ranges::zip_view` appearance in the standard the easy use of projection for *Tuple-Like* objects might become even more important because its dereferenceable type is exactly *Tuple-Like*.

§ 1.4. Radix sort use case

Counting-based sorts, and Radix Sort in particular, provide another motivating use case. Today it is not possible to have a C++ standard conformant implementation that uses Radix Sort algorithm underneath because the complexity of `std::sort` is defined as the number of comparator calls, while counting-based sorts do not use a comparator at all.

However, the industry needs Radix Sort for performance reasons. Implementations of C++ standard parallel algorithms, such as oneAPI Data Parallel C++ Library (oneDPL) and CUDA Thrust, use Radix Sort conditionally under the hood of `std::sort`, checking data types of the input and the comparator. In this case, a special comparator is of no help to sort values by keys, and projections seem the only viable option.

That makes the proposed API applicable wider than just with the C++ standard library use cases.

§ 2. Proposed API

We propose the following API:

```
inline namespace /* unspecified */
{
    template <size_t I>
        inline constexpr /* unspecified */ get_element = /* unspecified */;
}
inline constexpr auto get_key = get_element<0>;
inline constexpr auto get_value = get_element<1>;
```

With that API the motivating use case code with the desired behavior would be:

```
std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}};
std::ranges::sort(v, std::less{}, std::get_element<0>);
```

or even

```
std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}};
std::ranges::sort(v, std::less{}, std::get_key);
```

Let's look at comparison tables (a.k.a. Tony Tables):

Comparison of proposed API with comparator-based version

Before	After
<pre>std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}}; std::ranges::sort(v, [](auto x, auto y) { return std::get<0>(x) < std::get<0>(y); // key-based sorting });</pre>	<pre>std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}}; std::ranges::sort(v, std::less{}, std::get_key);</pre>

Comparison of proposed API with projections-based version

Before	After
<pre>std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}}; std::ranges::sort(v, std::less{}, [](auto x) { return std::get<0>(x); // key-based sorting });</pre>	<pre>std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}}; std::ranges::sort(v, std::less{}, std::ranges::get_key);</pre>

§ 2.1. Possible implementation

```
namespace std
{
    namespace __detail
    {
        template <std::size_t _Ip>
        struct __get_element_fn
        {
            template <typename _TupleLike>
            auto operator()(_TupleLike&& __tuple_like) const ->
                decltype(get<_Ip>(std::forward<_TupleLike>(__tuple_like)))
            {
                return get<_Ip>(std::forward<_TupleLike>(__tuple_like));
            }
        };
    }
}
```

```

    }
};
} // namespace __detail

inline namespace __get_element_namespace
{
template <std::size_t _Ip>
inline constexpr __detail::__get_element_fn<_Ip> get_element;
} // inline namespace __get_element_namespace

inline constexpr auto get_key = get_element<0>;
inline constexpr auto get_value = get_element<1>;
} // namespace std

```

§ 2.2. tuple-like concept

With the proposed `std::get_element` CPO, the *tuple-like* concept can be generalized to cover wider range of types rather than just the listed standard types.

§ 2.2.1. tuple-like concept generalization with `get_element`

With `get_element` we can define an exposition only helper concept *can-get-tuple-element* in the following way:

```

// necessary to check if std::tuple_size_v is well-formed before using it
template <typename T>
concept /*has-tuple-size*/ = // exposition only
requires {
    typename std::tuple_size<T>::type;
};

template< class T, std::size_t N >
concept /*can-get-tuple-element*/ = // exposition only
/*has-tuple-size*/<T> &&
requires(T t) {
    requires N < std::tuple_size_v<T>;
    typename std::tuple_element_t<N, T>;
    { std::get_element<N>(t) } -> std::convertible_to<const std::tuple_element_t<N, T>&
};

```

Then the *tuple-like* concept can use *can-get-tuple-element* and do something like:

```

template <typename T>
concept /*tuple-like*/ = !std::is_reference_v<T> &&
/*has-tuple-size*/<T> &&
[]<std::size_t... I>(std::index_sequence<I...>) {
    return (... && /*can-get-tuple-element*/<T, I>);
} (std::make_index_sequence<std::tuple_size_v<T>>{});

```

§ 3. Design considerations

Alternative name for the proposed API could be `std::ranges::get`. Unfortunately, this name is already taken for `std::ranges::subrange` overload.

Potentially `std::ranges::get` could be repurposed for the proposed CPO with minimal API break for tricky scenarios only while still working as expected in existing reasonable use cases, as explained below. But we (likely) could not avoid an ABI break.

As `std::get_element` have got more support than `std::ranges::get` at § 8.1 SG9 polls, Issaquah 2023, the rest of this section is kept primarily for recording the evaluated and rejected alternative.

§ 3.1. What could be done to use `std::ranges::get` name

In all major standard library implementations (GCC, LLVM, Microsoft) the `get` overload for `std::ranges::subrange` is defined in `std::ranges`. Adding another definition of `get` to the same namespace would obviously create name ambiguity.

However, library implementors could move the current `std::ranges::get` function to an implementation specific namespace (e.g., `__detail`) and inherit (possibly privately) `std::ranges::subrange` from an empty tag class (e.g., `adl_hook`) defined in the same namespace. That way `std::ranges::__detail::get` can still be found by ADL for `std::ranges::subrange`, and at the same time, the `std::ranges::get` name becomes free to be redefined as a CPO that could successfully find a necessary overload for `get`, including the case when the argument is a `std::ranges::subrange`. Moreover, the proposed `std::ranges::get` CPO type could have a parameter pack in the interface to cover the use case when the current `std::ranges::get` function is used with explicit template arguments.

Please see the example that explains the idea and shows how it might look like. A full implementation with examples is available [here](#).

```
namespace std
{
    namespace ranges
    {
        // Necessary to make namespace __detail being considered by ADL
        // for get<0>(std::ranges::subrange<something>{}) without moving
        // the subrange itself to another namespace
        namespace __detail
        {
            struct adl_hook {};
        }

        // thanks to the empty-base optimization, inheriting adl_hook does not break ABI
        template <class T> class subrange : __detail::adl_hook {
            public: T whatever;
        };

        namespace __detail
        {
            template <std::size_t, class T>
            auto get(subrange<T> x) {
                return x.whatever;
            }
        } // namespace __detail
    } // namespace ranges

    using std::ranges::__detail::get;
} // namespace std

namespace std
{
    namespace ranges
    {
        namespace __detail
        {
            // Introduce Args... to cover the case of calling get with explicit template arguments
            template <std::size_t _Ip, typename... Args>
            struct __get_fn
            {
                // No more than std::tuple_size_v template arguments should be allowed
                template <typename _TupleLike> requires (sizeof...(Args) <= std::tuple_size_v<std::remov
                    && __are_tuple_elements_convertible_to_args<std::remov
```



```

decltype(auto) operator()(_TupleLike&& __tuple_like) const
{
    return get<_Ip>(std::forward<_TupleLike>(__tuple_like));
}
};
} // namespace __detail

inline namespace __get_fn_namespace
{
template <std::size_t _Ip, typename... Args>
inline constexpr __detail::__get_fn<_Ip, Args...> get;
} // inline namespace __get_fn_namespace
} // namespace ranges
} // namespace std

```

With such an implementation, all important cases from our perspective continue working:

- `std::ranges::get<0>(sub_r)`
- `std::get<0>(sub_r)`
- `get<0>(sub_r)`
- `std::ranges::get<0, some_arg>(sub_r)` (can also be used without namespace)

where `sub_r` is `std::ranges::subrange` object.

The API breaking change appears when `get` has all explicit template arguments for `subrange`, i.e., `std::ranges::get<Iarg, Sarg, Karg>(std::ranges::subrange<Iarg, Sarg, Karg>{})`. The problem is with the last `Karg` argument, which is unrelated to `tuple_size_v` and `tuple_element_t` of the subrange. Even if we say that the proposed backward compatible CPO with `Args...` does not constraint `sizeof...(Args)` and ignores the tail outside `tuple_size_v<subrange>`, it doesn't help for the mentioned use case because `K` of `std::ranges::subrange` is a non-type template parameter. Anyway, this scenario doesn't look common because explicit template parameters are used relatively rarely and furthermore, `K` has the default argument that is substituted based on a sentinel.

Definitely such a change would break the ABI for `std::ranges::get` because the fully qualified name of this function would change from what is in C++ standard library implementations today. But we think that that ABI break would have low impact because `std::ranges::get` is likely inlined and so doesn't create a visible symbol in translation units. We could imagine other tricky scenario where the API might be broken when `std::ranges::get` is used for something else but call. For example: `&std::ranges::get<0>`, `decltype(std::ranges::get<0>)`, etc. However, these scenarios don't look common.

Since the `std::ranges::subrange` API is relatively new, perhaps only a small amount of users would be affected but it can not be predicted accurately.

§ 4. Connections with other papers

§ 4.1. Connection with [P2547R1]

[P2547R1] uses `std::get` as the example and a good candidate to be a customizable function. Authors plan to ship the customizable functions proposal first and deal with customizing standard library functions later. That means we should not expect that examples in this paper automatically would be transformed to customizable functions when it will land.

Moreover, at this time the authors of [P2547R1] don't see how to introduce customizable functions with the same names (e.g. `std::get`) without the ABI break, so they will likely need to choose different names.

§ 4.2. Connection with [P2141R1]

[P2141R1]'s main goal is allow aggregates being interpreted as *Tuple-Like*. At the same time, it touches the *tuple-like* concept making it as generic as for the types structured binding can work with. It also adds a yet another `std::get` overload that works with any *Tuple-Like* object except those that are already in the `std::` namespace.

With [P2141R1] being adopted `std::get` does the right thing and works with *Tuple-Like* object, so we may use just `std::get<_Ip>(std::forward<_TupleLike>(__tuple_like))` within the implementation of `std::get_element` instead of the unqualified `get` call.

Independently of [P2141R1] `std::get_element` brings its own value by covering the described motivation use-cases. Furthermore, in the standard there are already precedences of having two similar things with slightly different semantics, for example, `std::less` and `std::ranges::less`, where the latter is not even a CPO.

[P2141R1] also gives another way to generalize the *tuple-like* concept (via structured binding).

5. Further work

- Substitute `std::get_element` for `std::get` in formal wording of the APIs with *tuple-like*
- Broader implementation experience

6. Formal wording

Below, substitute the  character with a number the editor finds appropriate for the table, paragraph, section or sub-section.

6.1. Modify Concept *tuple-like* [tuple.like]

```
template<typename T>
concept has-tuple-size = // exposition only
    requires {
        typename tuple_size<T>::type;
    };

template<class T, size_t N>
concept can-get-tuple-element = // exposition only
    has-tuple-size<T> &&
    requires(T t) {
        requires N < std::tuple_size_v<T>;
        typename std::tuple_element_t<N, T>;
        { std::get_element<N>(t) } -> std::convertible_to<const std::tuple_element_t<N, T>&
    };

```

```
template<typename T>
concept tuple-like = see below
    !is_reference_v<T> &&
    has-tuple-size<T> &&
    [ ]<size_t... I>(index_sequence<I...>) {
        return (... && ranges::can-get-tuple-element<T, I>);
    } (make_index_sequence<tuple_size_v<T>>{});

```

A type `T` models and satisfies the exposition-only concept *tuple-like* if `remove_cvref_t<T>` is a specialization of `array`, `complex`, `pair`, `tuple`, or `ranges::subrange`.

§ 6.2. Modify Header `<tuple>` synopsis [[tuple.syn](#)]

```
[...]
// [tuple.helper], tuple helper classes
template <class T>
    constexpr size_t tuple_size_v = tuple_size<T>::value;

inline namespace /* unspecified */ {
    template <size_t I>
        inline constexpr /* unspecified */ get_element = /* unspecified */;
}
inline constexpr auto get_key = get_element<0>;
inline constexpr auto get_value = get_element<1>;
```

§ 6.3. Add the following sections into [[tuple](#)]

```
[...]
◆ Element access [tuple.elem]
◆ Customization Point Objects [tuple.cust]
◆ get_element [tuple.cust.get\_elem]
```

§ 6.4. Add the following wording into [[tuple.cust.get_elem](#)]

1. The name `get_element` denotes a customization point object ([\[customization.point.object\]](#)).
The expression `get_element<I>(E)` where `I` is `size_t` for a subexpression `E` is expression-equivalent to:
 1. `get<I>(E)`, if `E` has class or enumeration type and `get<I>(E)` is a well-formed expression when treated as an unevaluated operand, where the meaning of `get` is established as-if by performing argument-dependent lookup only ([\[basic.lookup.argdep\]](#)).
 2. Otherwise, `get_element<I>(E)` is ill-formed.

§ 6.5. Add feature test macro to the end of [[version.syn](#)]

```
[...]
#define __cpp_lib_element_access_customization_point 200000L
// also in <tuple>, <utility>, <array>, <ranges>

[...]
```

§ 6.6. Modify tuple construct [tuple.cnstr]

```
template<tuple-like UTuple>
constexpr explicit(see below) tuple(UTuple&& u);
```

Let I be the pack $0, 1, \dots, (\text{sizeof} \dots (\text{Types}) - 1)$.

Constraints:

- `different-from<UTuple, tuple> ([range.utility.helpers])` is true,
- `remove_cvref_t<UTuple>` is not a specialization of `ranges::subrange`,
- `sizeof... (Types)` equals `tuple_size_v<remove_cvref_t<UTuple>>`,
- `(is_constructible_v<Types, decltype(get_element<I>(std::forward<UTuple>(u)))> && ...)` is true, and
- either `sizeof... (Types)` is not 1, or (when `Types...` expands to `T`) `is_convertible_v<UTuple, T>` and `is_constructible_v<T, UTuple>` are both false.

Effects: For all i , initializes the i th element of `*this` with `get_element<i>(std::forward<UTuple>(u))`.

Remarks: The expression inside `explicit` is equivalent to: `!(is_convertible_v<decltype(get_element<I>(std::forward<UTuple>(u))), Types> && ...)` The constructor is defined as deleted if `(reference_constructs_from_temporary_v<Types, decltype(get_element<I>(std::forward<UTuple>(u)))> || ...)` is true.

§ 6.7. Modify tuple assignment [tuple.assign]

```
template <tuple-like UTuple>
constexpr tuple& operator=(UTuple&& u);
```

Constraints:

- `different-from<UTuple, tuple> ([range.utility.helpers])` is true,
- `remove_cvref_t<UTuple>` is not a specialization of `ranges::subrange`,
- `sizeof... (Types)` equals `tuple_size_v<remove_cvref_t<UTuple>>`, and,
- `is_assignable_v<Ti&, decltype(get_element<i>(std::forward<UTuple>(u)))>` is true for all i .

Effects: For all i , assigns `get_element<i>(std::forward<UTuple>(u))` to `get_element<i>(*this)`.

Returns: `*this`.

```
template<tuple-like UTuple>
constexpr const tuple& operator=(UTuple&& u) const;
```

Constraints:

- `different-from<UTuple, tuple> ([range.utility.helpers])` is true,
- `remove_cvref_t<UTuple>` is not a specialization of `ranges::subrange`,
- `sizeof... (Types)` equals `tuple_size_v<remove_cvref_t<UTuple>>`, and,
- `is_assignable_v<const Ti&, decltype(get_element<i>(std::forward<UTuple>(u)))>` is true for all i .

Effects: For all i , assigns `get_element<i>(std::forward<UTuple>(u))` to `get_element<i>(*this)`.

Returns: `*this`.

§ 6.8. Modify `tuple_cat` in tuple creation [[tuple.creation](#)]

```
template<tuple-like... Tuples>
constexpr tuple<CTypes...> tuple_cat(Tuples&&... tpls);
```

Let n be `sizeof...(Tuples)`. For every integer $0 \leq i < n$:

- Let T_i be the i th type in `Tuples`.
- Let U_i be `remove_cvref_t<T_i>`.
- Let tp_i be the i th element in the function parameter pack `tpls`.
- Let S_i be `tuple_size_v<U_i>`.
- Let $E_{k,i}$ be `tuple_element_t<k, U_i>`.
- Let eki be `get<element<k>(std::forward<T_i>(tp_i))>`.
- Let $Elems_i$ be a pack of the types $E_{0,i}, \dots, E_{S_i-1,i}$.
- Let $elems_i$ be a pack of the expressions $e_{0,i}, \dots, e_{S_i-1,i}$.

The types in `CTypes` are equal to the ordered sequence of the expanded packs of types $Elems_0, \dots, Elems_1, \dots, Elems_{n-1}$. Let $celems$ be the ordered sequence of the expanded packs of expressions $elems_0, \dots, elems_{n-1}$.

Mandates: `(is_constructible_v<CTypes, decltype(celems)> && ...)` is true.

Returns: `tuple<CTypes...>(celems...)`

§ 6.9. Modify `apply` in [[tuple.apply](#)]

```
template<class F, tuple-like Tuple>
constexpr decltype(auto) apply(F&& f, Tuple&& t) noexcept(see below);
```

Effects: Given the exposition-only function template:

```
namespace std {
    template<class F, tuple-like Tuple, size_t... I>
    constexpr decltype(auto) apply_impl(F&& f, Tuple&& t, index_sequence<I...>) {
        // exposition only
        return INVOKE(std::forward<F>(f), get<element<I>(std::forward<Tuple>(t))...>);
    }
}
```

Equivalent to:

```
return apply_impl(std::forward(f), std::forward(t),
    make_index_sequence<tuple_size_v<remove_reference_t<Tuple>>-1>());
```

Remarks: Let I be the pack $0, 1, \dots, (\text{tuple_size_v}<\text{remove_reference_t}<\text{Tuple}> - 1)$. The exception specification is equivalent to:

```
noexcept(invoker(std::forward<F>(f), get<element<I>(std::forward<Tuple>(t))...))
```

```
template<class T, tuple-like Tuple>
    constexpr T make_from_tuple(Tuple&& t);
```

Mandates: If `tuple_size_v<remove_reference_t<Tuple>>` is 1, then
`reference_constructs_from_temporary_vT`, `decltype(get_element<0>(declval<Tuple>()))` is
false.

Effects: Given the exposition-only function template:

```
namespace std {
    template<class T, tuple-like Tuple, size_t... I>
        requires is_constructible_v<T, decltype(get_element<I>(declval<Tuple>()))...>
        constexpr T make-from-tuple-impl(Tuple&& t, index_sequence<I...>) { // exposition only
            return T(get_element<I>(std::forward<Tuple>(t))...);
        }
}
```

§ 6.10. Modify relation operators in [tuple.rel]

```
template<class... TTypes, class... UTypes>
constexpr bool operator==(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
template<class... TTypes, tuple-like UTuple>
constexpr bool operator==(const tuple<TTypes...>& t, const UTuple& u);
```

For the first overload let UTuple be tuple<UTypes...>.

Constraints: For all i , where $0 \leq i < \text{sizeof}...(TTypes)$, $\text{get}<i>(t) == \text{get_element}<i>(u)$ is a valid expression and $\text{decltype}(\text{get}<i>(t) == \text{get_element}<i>(u))$ models boolean-testable. $\text{sizeof}...(TTypes)$ equals $\text{tuple_size_v}<UTuple>$.

Returns: true if $\text{get}<i>(t) == \text{get_element}<i>(u)$ for all i , otherwise false.

[Note 1: If $\text{sizeof}...(TTypes)$ equals zero, returns true. — end note]

Remarks:

- The elementary comparisons are performed in order from the zeroth index upwards. No comparisons or element accesses are performed after the first equality comparison that evaluates to false.
- The second overload is to be found via argument-dependent lookup ([basic.lookup.argdep]) only.

```
template<class... TTypes, class... UTypes>
constexpr common_comparison_category_t<synth-three-way-result<TTypes, UTypes>...>
operator<=>(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
template<class... TTypes, tuple-like UTuple>
constexpr common_comparison_category_t<synth-three-way-result<TTypes, Elms>...>
operator<=>(const tuple<TTypes...>& t, const UTuple& u);
```

For the second overload, Elms denotes the pack of types $\text{tuple_element_t}<0, UTuple>$, $\text{tuple_element_t}<1, UTuple>$, ..., $\text{tuple_element_t}<\text{tuple_size_v}<UTuple> - 1, UTuple>$.

Effects: Performs a lexicographical comparison between t and u . If $\text{sizeof}...(TTypes)$ equals zero, returns $\text{strong_ordering}::\text{equal}$.

Otherwise, equivalent to: `if (auto c = synth-three-way(get<0>(t), get_element<0>(u)); c != 0) return c; return rtail <=> utail;`

where rtail for some r is a tuple containing all but the first element of r .

Remarks: The second overload is to be found via argument-dependent lookup ([basic.lookup.argdep]) only.

§ 6.11. Modify [range.elements.iterator]

The member *typedef-name* `iterator_category` is defined if and only if *Base* models `forward_range`. In that case, `iterator_category` is defined as follows: Let C denote the type `iterator_traits<iterator_t<Base>>::iterator_category`.

- If `std::get_element<N>(*current_)` is an rvalue, `iterator_category` denotes `input_iterator_tag`.
- Otherwise, if C models `derived_from<random_access_iterator_tag>`, `iterator_category` denotes `random_access_iterator_tag`.
- Otherwise, `iterator_category` denotes C .

```
static constexpr decltype(auto) get_element(const iterator_t<Base>& i);
```

Effects: Equivalent to:

```
if constexpr (is_reference_v<range_reference_t<Base>>) {
    return std::get_element<N>(*i);
} else {
```

```

using E = remove_cv_t<tuple_element_t<N, range_reference_t<Base>>>;
return static_cast<E>(std::get_element<N>(*i));
}

```

§ 7. Revision history

§ 7.1. R1 => R2

- Add extra motivation to allow user-defined tuples in the standard
- Propose changes to *tuple-like* concept with wording

§ 7.2. R0 => R1

- Address the "structured binding unused variables" questions with [\[P2169R3\]](#)
- Add ruminations about possible relaxation of the *tuple-like* concept
- Add connections to [\[P2547R1\]](#) and [\[P2141R1\]](#)
- Apply an approach to minimize ABI and API breaking for the `std::ranges::get` name
- Add wording and a feature test macro for the `std::get_element` API

§ 8. Polls

§ 8.1. SG9 polls, Issaquah 2023

POLL: The solution proposed in the paper "P2769: `get_element` customization point object" should be renamed to `std::ranges::get`.

SF	F	N	A	SA
1	2	1	2	1

POLL: The solution proposed in the paper "P2769: `get_element` customization point object" should be moved out of the `ranges` namespace (`std::get_element`).

SF	F	N	A	SA
2	4	0	1	0

§ 8.2. Library Evolution Telecon 2024-01-23

POLL: [\[P2769R1\]](#) (`get_element` customization point object) needs to allow for user tuple-likes before it can ship

SF	F	N	A	SA
3	3	4	2	0

POLL: LEWG should spend more time on [\[P2769R1\]](#) (`get_element` customization point object)

SF	F	N	A	SA
5	4	0	2	0

§ 9. Acknowledgements

- Thanks to Casey Carter for providing `adl_hook` trick for [§ 3 Design considerations](#)
- Thanks to Corentin Jabot for providing the input for [§ 4.1 Connection with \[P2547R1\]](#) and for the initial *tuple-like* concept relaxing idea
- Thanks to Benjamin Brock for paper review and design discussions.

§ References

§ Informative References

[P2141R1]

Antony Polukhin. *Aggregates are named tuples*. 3 May 2023. URL: <https://wg21.link/p2141r1>

[P2165R4]

Corentin Jabot. *Compatibility between tuple, pair and tuple-like objects*. 15 July 2022. URL: <https://wg21.link/p2165r4>

[P2169R3]

Corentin Jabot, Michael Park. *A Nice Placeholder With No Name*. 15 December 2022. URL: <https://wg21.link/p2169r3>

[P2547R1]

Lewis Baker, Corentin Jabot, Gašper Ažman. *Language support for customisable functions*. 16 July 2022. URL: <https://wg21.link/p2547r1>

[P2769R1]

Ruslan Arutyunyan, Alexey Kukanov. *get_element customization point object*. 17 May 2023. URL: <https://wg21.link/p2769r1>